Dense_Prediction

2D

1. CNN-based
    1. In computer vision pixelwise dense prediction is the task of predicting a label for each pixel in the image https://arxiv.org/abs/1611.09288.
        1. Semantic segmentation: classify per pixel
        2. Depth estimation: regress per pixel.
        3. Instance segmentation: classify per pixel per object.
        4. Panoptic segmentation: handles both stuff (semantic segmentation) and things (instance segmentation).
            1. Assign (category, instance_id) pair to each pixel in image.
            2. Instance label ignored for "stuff" categories.
    2. FCN for semantic segmentation
        1. Why not stack convolutions? suppose 200 3x3 filters/layer, H=W=400. Storage/layer/image: 200 *400* 400 * 4 bytes = 122MB, 122MB*100 layers, batch size of 20 = 238 GB of memory. [2]
        2. Key idea: encoder-decoder, first downsample towards middle of network, then upsample from middle,
        3. how do we downsample? convolutions, pooling.
        4. Putting it together: conv+pooling downsample/compress/encode, Transpose convs/unpoolings upsample/uncompress/decode.
    3. U-Net for semantic segmentation and depth estimation
    4. Mask R-CNN for instance segmentation
        1. Boxes first paradigm:
            1. Run detector (Faster-RCNN).
            2. Produce segmentation relative to each predicted box.
        2. Mask R-CNN combines both steps into an end-to-end trainable model.
2. Transformer
    1. Dense Prediction Transformer (DPT) for semantic segmentation and depth estimation
    2. SETR (semantic segmentation transformer)

3D

1. semantic segmentation:
    1. Point cloud: PointNet
        1. n points have 3 dimensions.
        2. There are 2 important parts in pointnet:
            1. Apply MLP from 3 to 64 dimensions
                1. Then from 64 to 64 dimensions.
                2. get $n \times 64$ feature.
                3. only extract feature from each point, but not global.
            2. T-Net (to be invariant to permutations), to extract some global features from a set of point cloud features or point cloud coordinates:
                1. We should have a function that is symmetric to each point
                2. If $g(h(input)) = \gamma(g(h(x_1), h(x_2), \ldots, h(x_n)))$, then although we permute the input, the output will not change if the function is symmetric
                3. In this paper max_pooling is the best choice for the symmetric function
                4. input->MLP->max->MLP, h=MLP, g=max, $\gamma = \mathrm{MLP}$.
            3. T-Net (Input alignment by transformer network (to be invariant to any kind of rotation and transformation). T-Net is actually a simpler version of Spatial Transformer Network by Jaderberg et al.
                1. After the points goes through the T-Net, we can get some global features. We can use the global features to predict some inverse rotation matrix to make sure our input data can be inversely rotated into our canonical our original camera pose. This is the main motivation of input alignment by T-Net output.
                2. the transformation is just matrix multiplication.
                3. $\mathrm{matmul}(n \times 3, 3 \times 3)$. The 3×3 transformation matrix is output of T-Net.

4. Then apply 3 layer MLP for local embedding, then get $n \times 1024$
5. Apply max pooling to the $n \times 1024$ local embedding to get global feature, global feature size is 1×1024.
6. After getting local features, use extra MLPs to perform classification that outputs k scores by applying a softmax and cross-entropy to supervise the whole network.

2. Voxel: 3D U-Net

2. instance segmentation: SoftGroup

References:

1. CSEP 576: Dense prediction, Udub, Jonathan Huang, 2020.
2. CSEP 576: Deep learning in 3D, Udub, Vita Ablavsky, 2021.